



MULTIMEDIA-PROGRAMAZIOA ETA GAILU MUGIKORRAK ANDROID - SARRERA

1. AURKIBIDEA

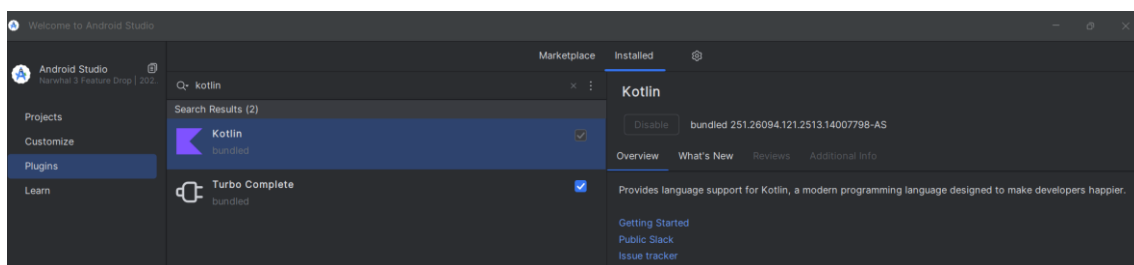
1. Ingurunearen instalazioa (Android Studio)
2. Lehenengo proiektua sortu.
3. Proiektu baten estructurea
4. Aplikazio baten osagaiak
5. MainActivity aztertzen

2. INGURUNEAREN INSTALAZIOA (ANDROID STUDIO)

Lehenik eta behin Android Studioren azken bertsioiko egonkorra instalazioa burutuko dugu:

<https://developer.android.com/studio>

Android Studio 4.1 bertsiotik aurrera **Kotlin** plugina instalatuta agertzen da jada, beraz, ez dugu eskuz instalatu behar. Azken bertsioa instalatuta duzula ziurtatzeko.



Horrez gain erronkarako beharrezkoa izango duzuen JSON klasea instalatu.

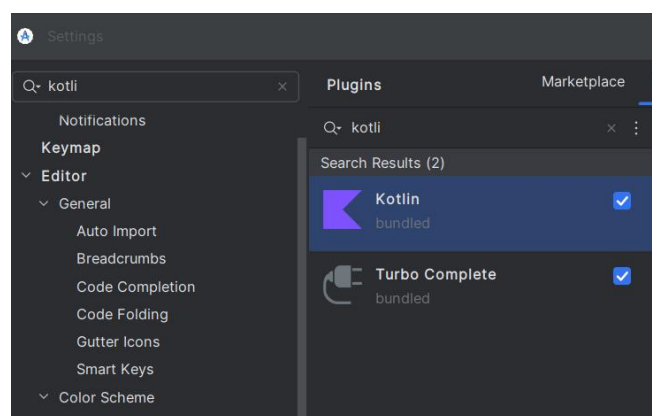
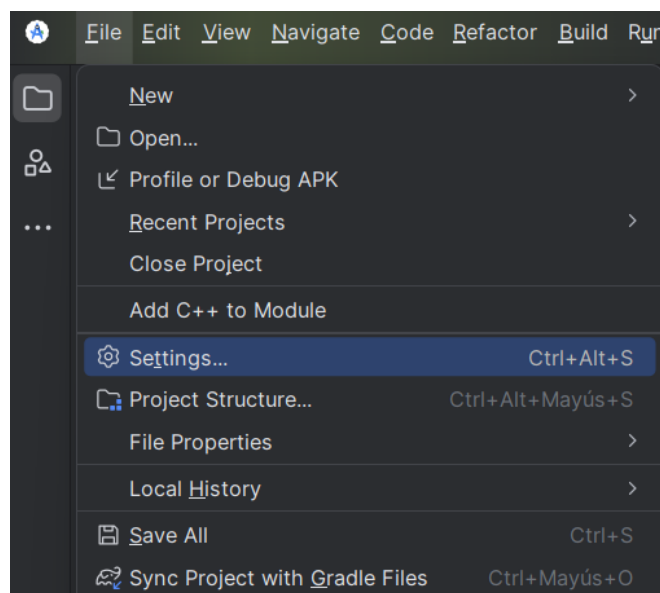


Behin Android Studio barruen egonda, plugin-konfigurazioen atala:

File → Settings...

Plugins

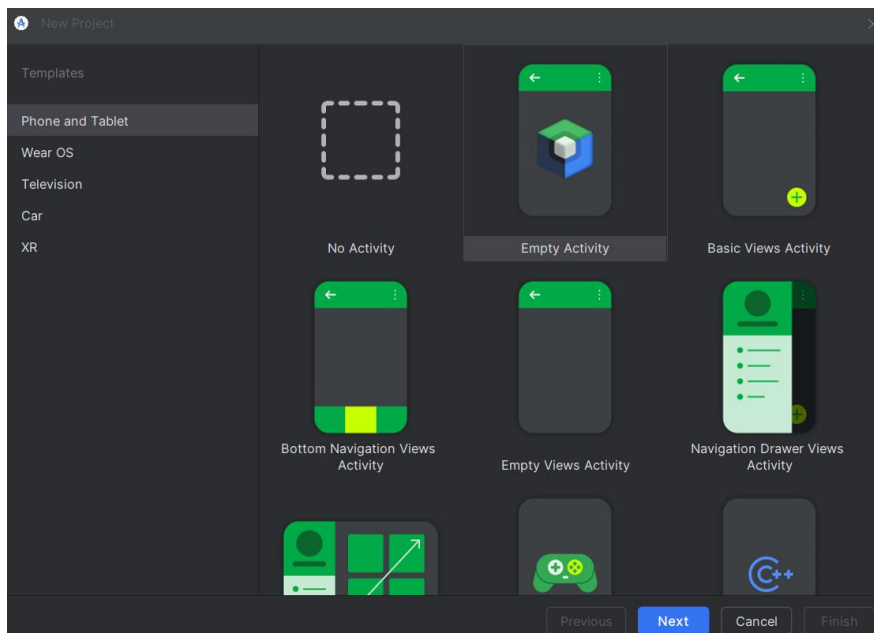
Menuan Kotlin idatzi



3. LEHENENGO PROIEKTUA SORTU.

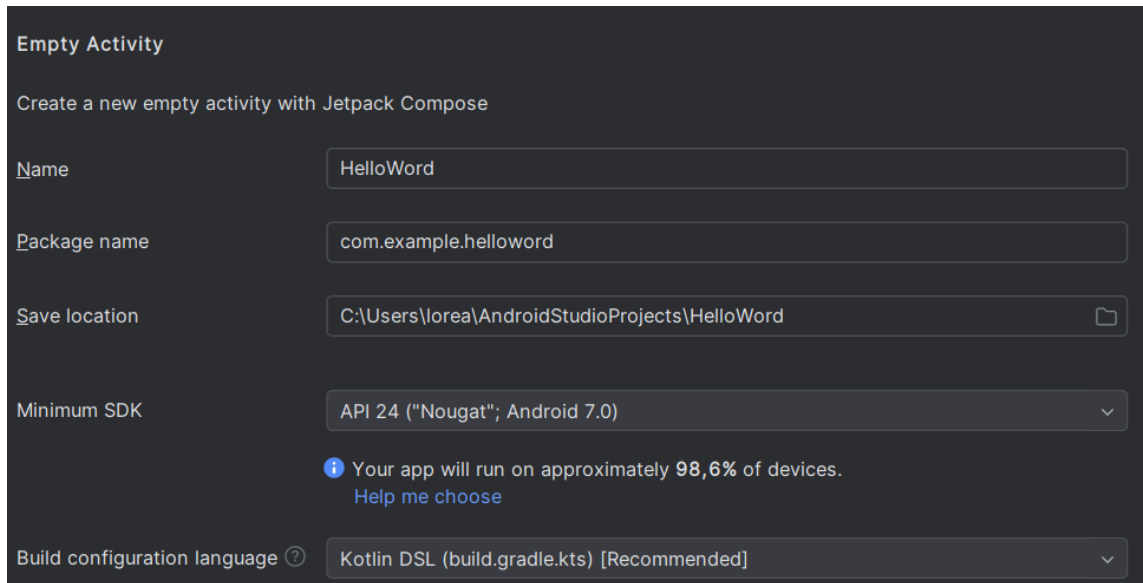
Oso erraza da. Android Studio 3.0 bertsiotik aurrera, **Kotlin** erabat integratuta dago ingurunean, beraz, zuzenean sor ditzakezu zure proiektuak Kotlinen. Ez du inolako misteriorik. Hala ere, ikusiko dugu nola konfiguratu eskuz, oso erabilgarria izango dena Javan proiektu bat baduzu eta bertan Kotlin erabiltzen hasi nahi baduzu.

File → New → New Project → Empty Activity



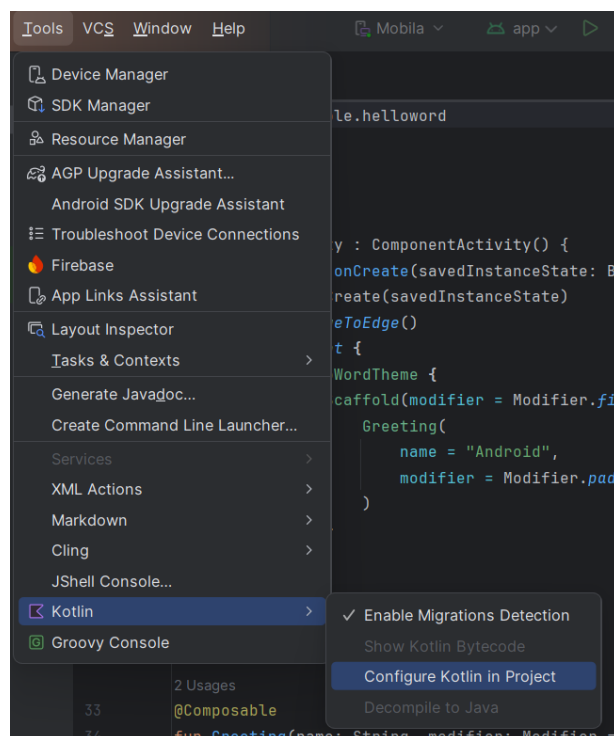
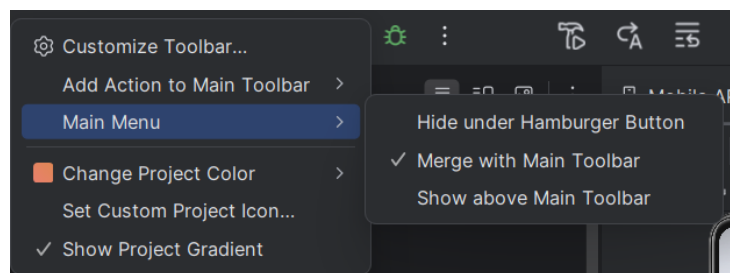
Pantaila honetan zure proiektua konfiguratuko da.

- 1.- Adierazi nahi duzun **izena** eta **kokapena**.
- 2.- **Androiden bertsio** minimoa ere nahi duzuna izan daiteke. Oro har, erabiltzaileen % 85 inguru bada, zifra ona da.
- 3.- Gainerakoa lehenetsita utz dezakezu.
- 4.- Sakatu **Finish**.



Kotlin-en konfigurazioa begitu nahi izanez gero:

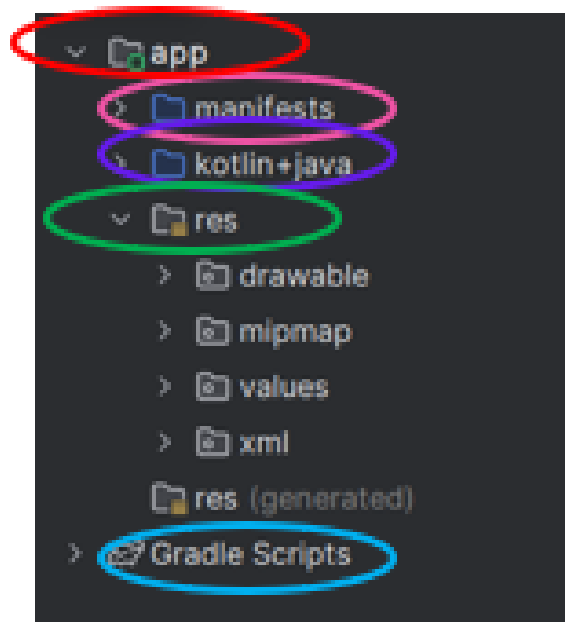
Menua bistaratzeko:



4. PROIEKTU BATEN ESTRUKTURA

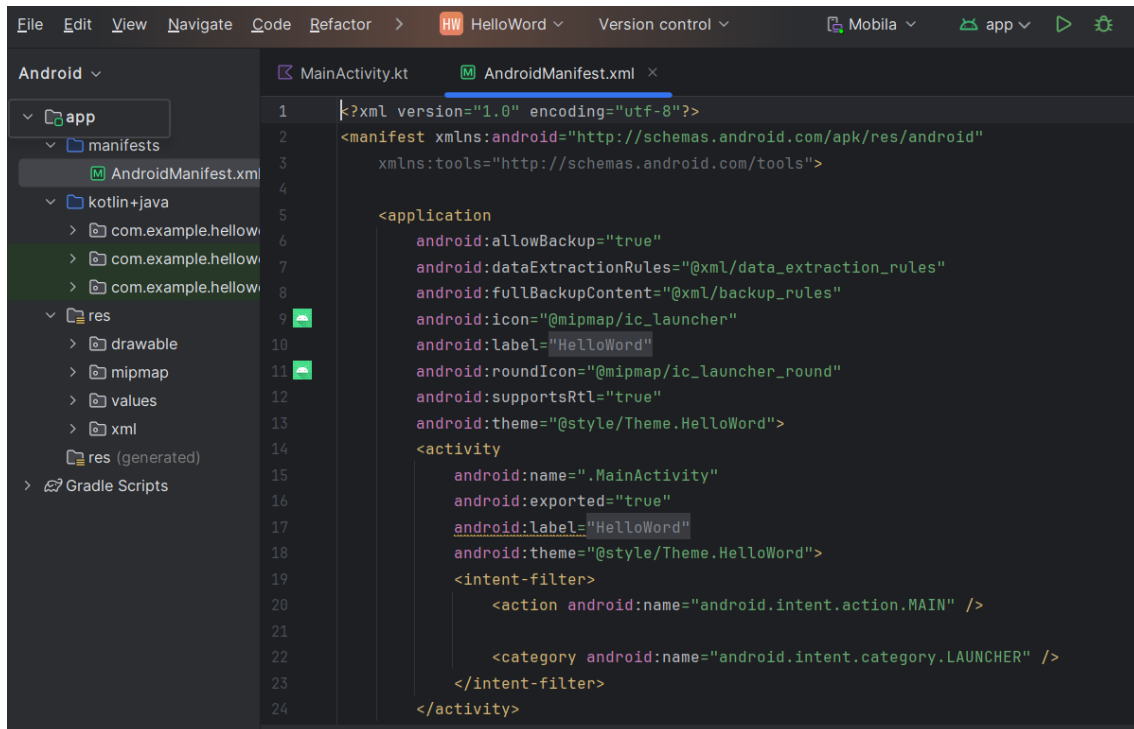
Proiektua fitxategi eta karpeta osatua dago:

- **Moduloa**
- **Aplikazioaren deskribatzailea**
- **Kodigoa**
- **Errekurtsoak**
- **Konpilazioko konfigurazioa**

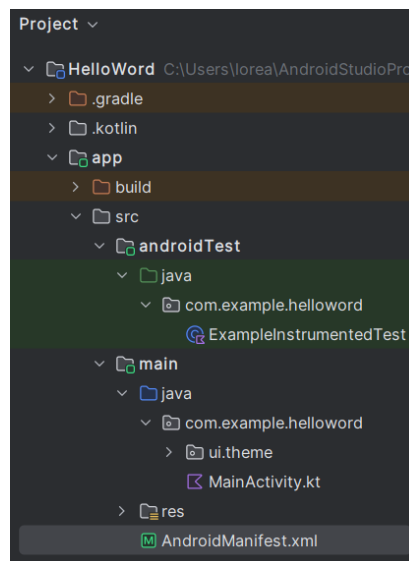


- **Aplikazioaren deskribatzailea (AndroidManifest.xml)**

Android aplikazioa deskribatzen du. Bertan, aplikazioaren jarduerak, barneratzeak, zerbitzuak eta edukiaren hornitzaileak adierazten dira. Halaber, aplikazioak eskatzen dituen baimenak deklaratzeko dira. Androiden bertsio minimoa adierazten da exekutatu ahal izateko, Java paketea, aplikazioaren bertsioa, etab.



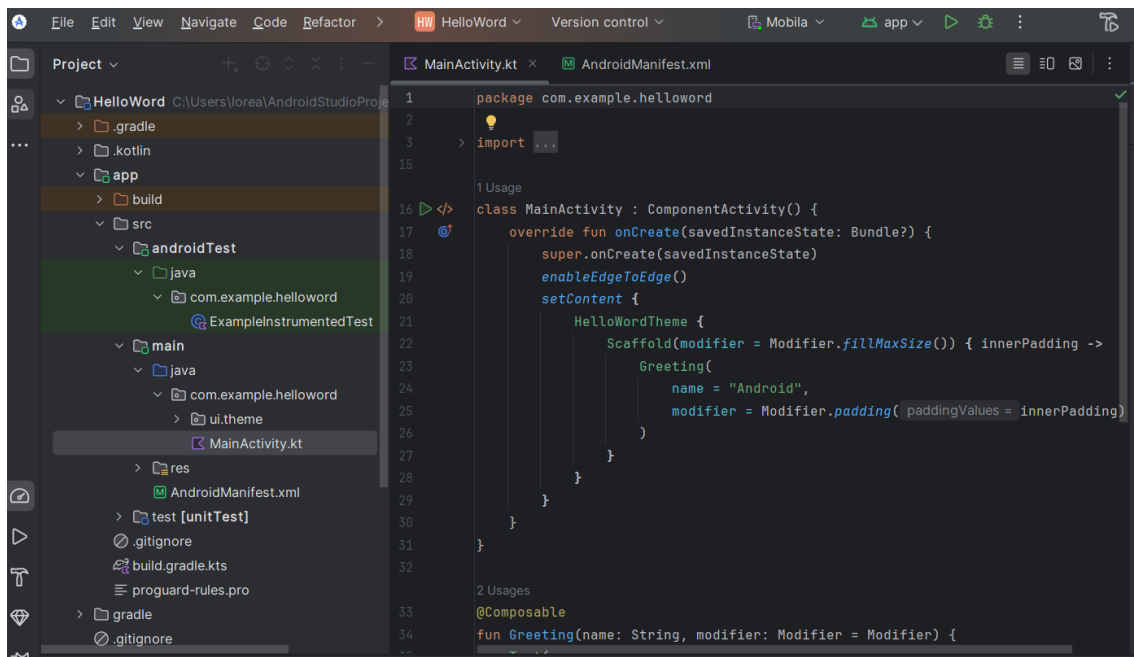
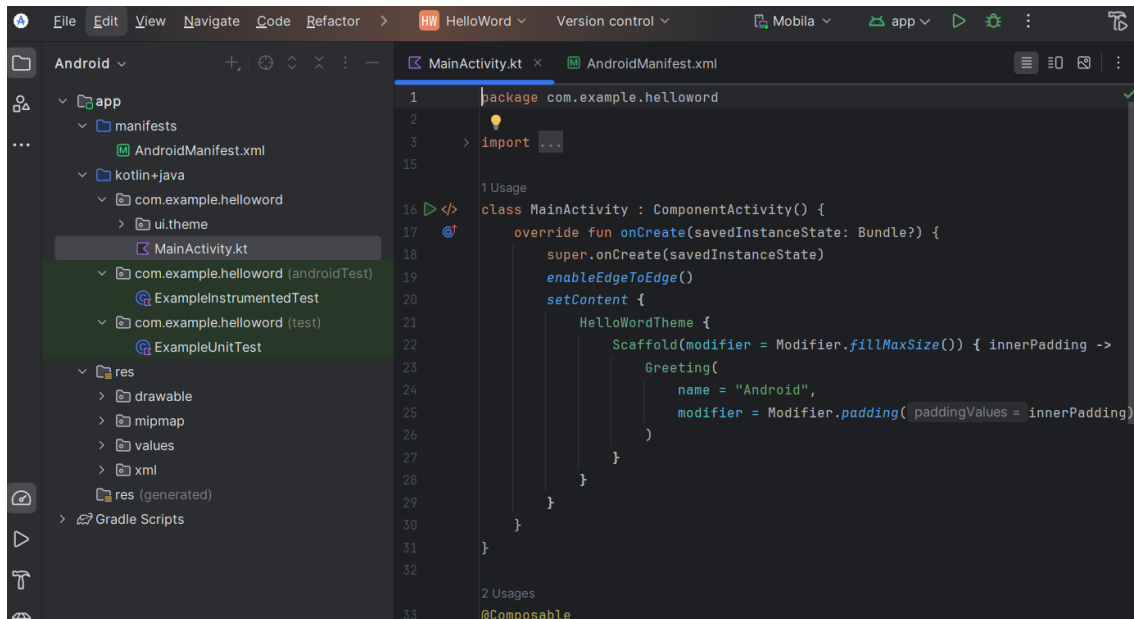
```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:tools="http://schemas.android.com/tools">
4
5     <application
6         android:allowBackup="true"
7         android:dataExtractionRules="@xml/data_extraction_rules"
8         android:fullBackupContent="@xml/backup_rules"
9         android:icon="@mipmap/ic_launcher"
10        android:label="@string/app_name"
11        android:roundIcon="@mipmap/ic_launcher_round"
12        android:supportRtl="true"
13        android:theme="@style/Theme.HelloWord">
14
15        <activity
16            android:name=".MainActivity"
17            android:exported="true"
18            android:label="@string/app_name"
19            android:theme="@style/Theme.HelloWord">
20            <intent-filter>
21                <action android:name="android.intent.action.MAIN" />
22                <category android:name="android.intent.category.LAUNCHER" />
23            </intent-filter>
24        </activity>
25    </application>
26</manifest>
```



- **Kodigoa (Kotlin + Java)**

Aplikazioaren iturburu-kodea duen karpeta. Paketearen izenaren arabera gordetzen da karpetetan.

- **MainActivity**: Java klasea hasierako jardueraren kodearekin.
- **ApplicationTest**: JUnit API erabiliz aplikazioaren testatze-kodea txertatzeko pentsatutako Java klasea.



- Errekurtsioak (res)

Aplikazioak erabiltzen dituen baliabideak biltzen dituen karpeta.

Drawable: karpeta horretan gordetzen dira irudi-fitxategiak (jpg edo png) eta irudi-deskribatzaileak XMLn (atzizki berezi bat erabiliko dugu baliabide baten bertsio bat baino gehiago izan nahi badugu).

Mipmap: drawableak bezala, baina sistemek ezin dituzte berriro eskalatu

Layout: Aplikazioaren bistako xml fitxategiak ditu. Bisten bidez, aplikazioaren erabiltzaile-interfazea osatuko duten pantailak konfiguratu ahal izango ditugu. Web-orrialdeak diseinatzeko erabilitako HTMLaren antzeko formatu bat erabiltzen da.

Menu: XML fitxategiak jarduera bakoitzeko menuekin.

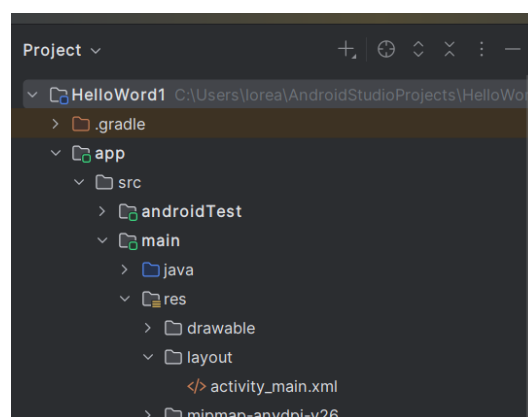
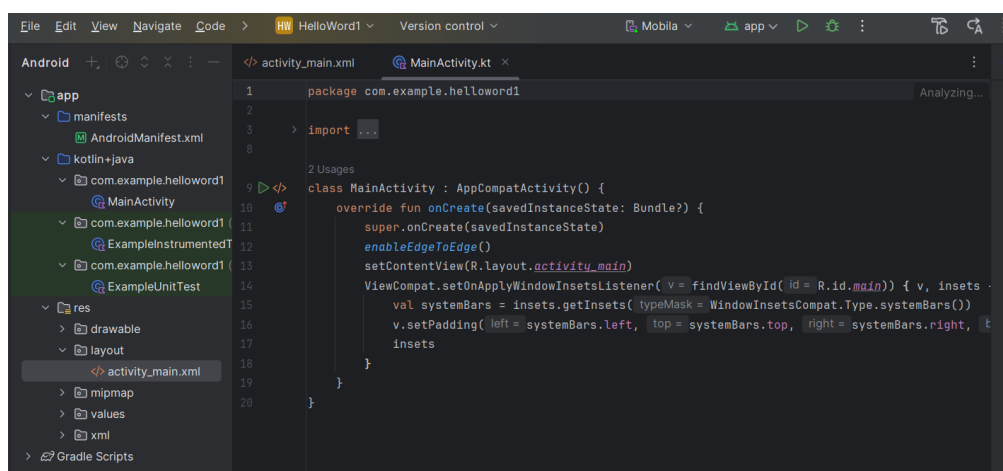
Values: XML fitxategiak ere erabiliko ditugu aplikazioan erabilitako balioak adierazteko. Horrela, fitxategi horietatik aldatu ahal izango ditugu iturburu-kodera joan barik.

colors.xml aplikazioan erabiliko diren koloreak definituko dira.

string.xml fitxategian, aplikazioaren karaktere-kate guztiak definitzen dira.

themes.xml aplikazioaren estiloak eta gaiak definitzen dira.

Baliabide alternatiboak sortuz, oso erraza izango da aplikazio bat beste hizkuntza batera itzultzea.

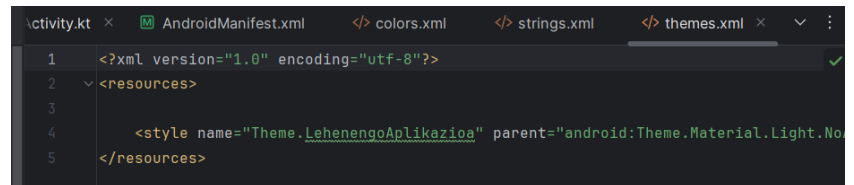
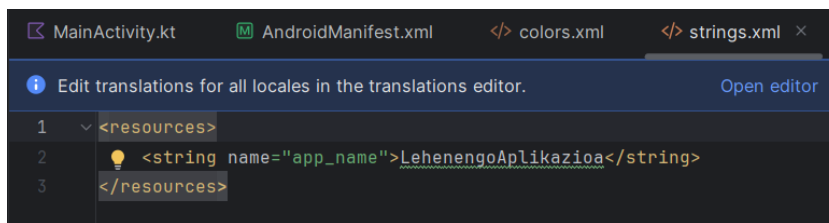
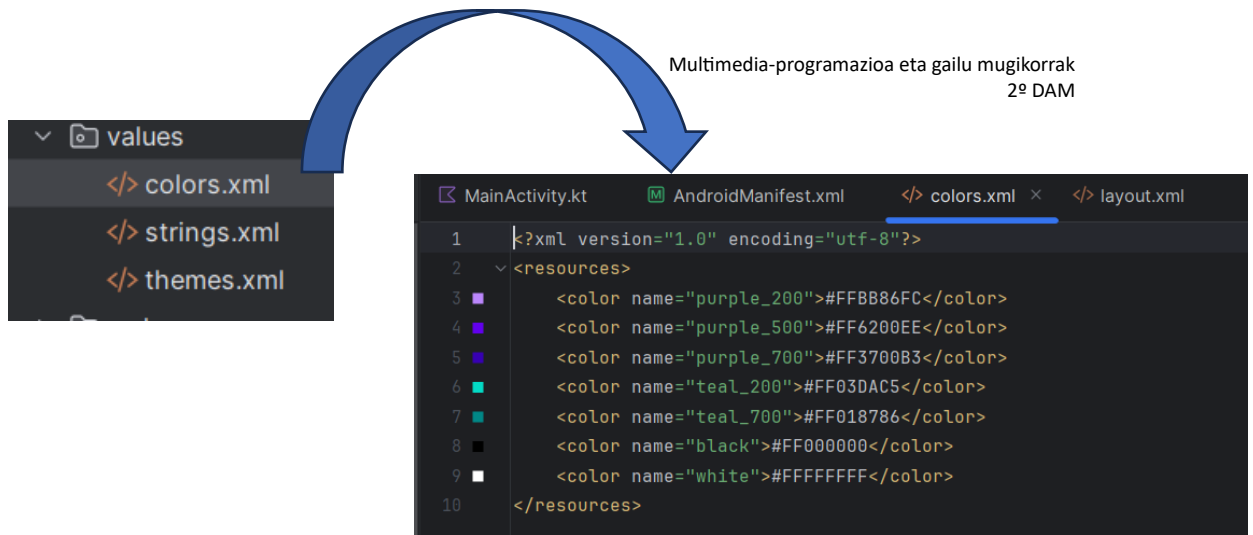


- **Konpilazioko konfigurazioa (Gradle Scripts)**

Karpeta honetan Gradle fitxategi batzuk gordetzen dira aplikazioa eraikitzeko. Zenbait alderdi garrantzitsu defini ditzakezue, hala nola konpilazioaren sdk bertsioa (targetSdkVersion) eta gutxienerako bertsioa (minSdkVersion).

Laburbilduz proiektuaren barruan, honako hauek aurkituko ditugu:

- **Jarduera nagusia:** Jarduera bat sortzen da, erabiltzaileari erakutsiko zaion lehena.
- **Ikusmenaren definizioa:** layout-aren barruan egotea
- **Baliabideen karpeta:**
 - **drawable:** irudi-fitxategiak eta irudi-deskribatzaileak (.png, .jpg, .xml)
 - **mipmap:** drawable bezala, baina sistemak ezin ditu berriz eskalatu
 - **layout:** XML fitxategiak aplikaziora begira. Aplikazioaren pantailak diseinatzeko aukera ematen du.
 - **value:** kate-balioak, koloreak eta estiloak...
 - **menua:** XML fitxategiak, aplikazioaren menuekin.
 - **anim:** animazio-deskribatzaileak dituzten XML fitxategiak
 - **xml:** aplikazioak eskatzen dituen beste XML fitxategi batzuk.
 - **raw:** XML formatuan ez dauden fitxero gehigarriak
 - **doc:** proiektuari lotutako dokumentazioa
- **Balioen definizioa**
 - Balioa magnitude bat definitzen duen baliabidea da
 - Zenbait kolore, testu-kate, estilo, dimentsio eta abarretan definitzen dira
 - Kodea eta edukia bereizteko aukera ematen du
 - Gainera aplikazioa konfiguratzea eta itzultzea errazten du



5. APLIKAZIO BATEN OSAGAIAK

JARDUERA (ACTIVITY)

Androideko aplikazio bat, bistartzeko oinarritzko elementu multzo batez osatuta egongo da, aplikazioaren pantailak izenez ezagutzen direnak. Android sisteman, elementu edo pantaila horietako bakoitzari jarduera(activity) esaten zaio. Haren funtzio nagusia erabiltzaile-interfazea sortzea da. Aplikazio batek hainbat jarduera behar izaten ditu erabiltzaile-interfazea sortzeko. Sortutako jarduerak independenteak izango dira, nahiz eta guztiek helburu komun baterako lan egin.

BISTAK

Aplikazio baten erabiltzaile-interfazea osatzen duten elementuaz osatuak daude. (Adibidez, botoia edo testu-sarrera bat) view klasearen ondorengoak dira eta, beraz, Java kodea erabiliz defini daitezke. Hala ere, ohikoena izango da bistak XML fitxategia erabiliz definitzea, eta sistemak fitxategi horretatik abiatuta guk nahi ditugun objektuak sortzea.

LAYOUT

Forma jakin bateko ikuspegi taldekatuen multzoa da. Hainbat layout-mota izango ditugu forma linealeko bistak antolatzeko, koadrikulatan edo bista bakoitzaren posizio absolutua adieraziz.

ZERBITZUA (SERVICE)

Erabiltzailearekin interakziorik egin beharrik gabe gauzatzen den prozesua da. Unix-eko deabru baten edo Windows-eko zerbitzu baten antzeko zerbait da. Androidetan, bi motatako zerbitzuak ditugu: zerbitzu lokalak, prozesu berean exekutatzen direnak, eta urruneko zerbitzuak, prozesu bereizian(separados) exekutatzen direnak.

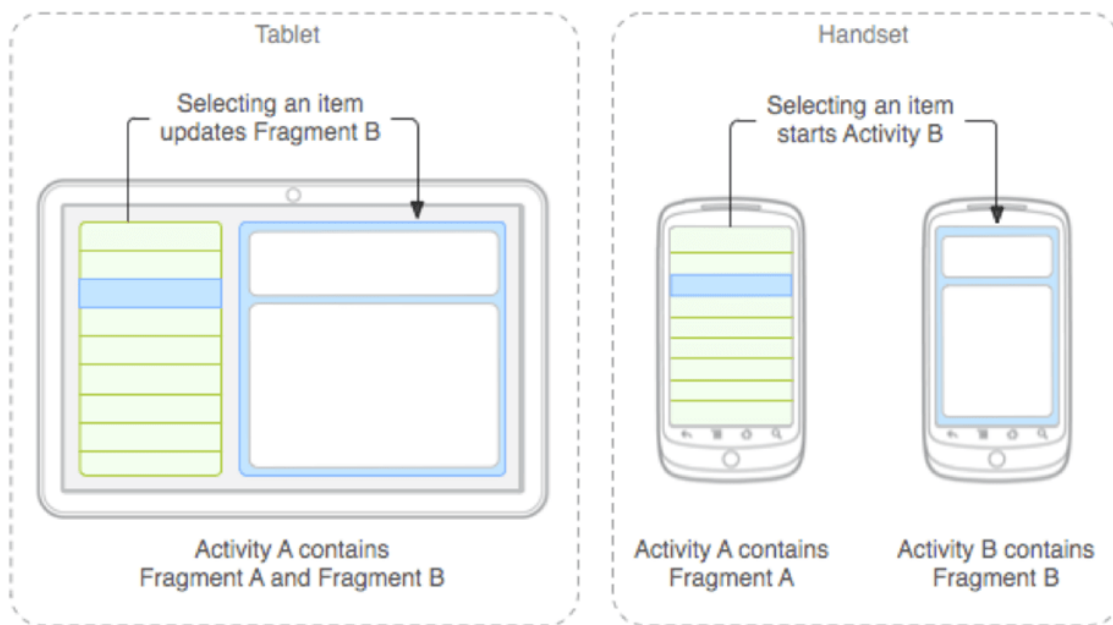
INTENT-AK

Ekintzaren bat egiteko aukera adierazten du; adibidez, telefono-dei bat egitea edota web-orri bat ikustea. Jarduera bat abiarazi, zerbitzu bat abiarazi, broadcast motako iragarki bat bidali edo zerbitzu batekin komunikatu nahi dugun bakoitzean erabiltzen da. Bidalitako osagaiak gure aplikazioaren barrukoak edo kanpokoak izan daitezke. Osagai horien artean informazioa trukatzeko ere erabiliko ditugu.

FRAGMENT

Tableten etorrerak arazoa berri bat ekarri zuen, android aplikazioek pantaila handiagoak jasan behar zituen. Gailu mugikor baterako pentsatutako aplikazio bat diseinatzen badugu eta gero tableta batean exekutatzen badugu, emaitza ez da aproposa izango.

Diseinatzaileari arazo hau konpontzen laguntzeko, Android-en 3.0 bertsioan fragmentak agertzen dira. Fragment bat hainbat bisten loturak osatzen du, erabiltzaile-interfazearen bloke funtzional bat sortzeko. Fragmentak sortu ondoren, jarduera(Activity) baten barruan fragment bat edo gehiago konbina ditzakegu, pantailaren tamainaren arabera.



IRAGARKIEN HARTZAILEA (BROADCAST RECEIVER)

Iragarkien hargailu batek broadcast motako iragarkien aurrean jaso eta erreakzionatzen du. Broadcast iragarkiak sistemaren edo aplikazioen arabera antola daitezke. Sistemak sortutako iragarki mota batzuk honako hauek dira: bateria baxua, sarrera deia... Aplikazioek broadcast iragarki mota berriak sortu eta merkaturatu ditzakete. Iragarkien hartzaileek ez dute erabiltzaile-interfazerik, baina jarduera bat has dezakete egokituz jotzen badute.

EDUKI-HORNITZAILEAK (CONTENT PROVIDER)

Askotan, Android terminal batean instalatutako aplikazioek informazioa partekatu behar izaten dute. Android-ek mekanismo estandar bat definitzen du, aplikazioek datuak partekatu ahal izan ditzaten fitxategi-sistemaren segurtasuna arriskuan jarri gabe. Mekanismo horren bidez, beste aplikazio batzuetako datuetara sartu ahal izango gara, hala nola kontaktu-zerrendara edo beste aplikazio batzuei datuak ematera.

6. MainActivity AZTERTZEN

```
class MainActivity : ComponentActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        enableEdgeToEdge()  
        setContent {  
            HelloWorldTheme {  
                Scaffold(modifier = Modifier.fillMaxSize()) { innerPadding ->  
                    Greeting(  
                        name = "Android",  
                        modifier = Modifier.padding(paddingValues = innerPadding)  
                    )  
                }  
            }  
        }  
    }  
}
```

class MainActivity : ComponentActivity()

- Aplikazioko **aktibitate nagusia** definitzen da, ComponentActivity-tik heredatuz. Honek Android-en bertsio zaharrei eta UI modernoari laguntza ematen die.

override fun onCreate(savedInstanceState: Bundle?)

- Metodo hau aktibitatea sortzen denean deitzen da. Hau da UI eta abio-logika iniciala konfiguratzekeko puntua.

super.onCreate(savedInstanceState)

- Superklaseko onCreate deitzen da, aktibitatearen konfigurazio basea ondo egin dadin.

enableEdgeToEdge()

- Honen helburua aplikazioa **egoera-barra eta nabigazio-barra azpian** marrazteko aukera ematea da, hau da, "edge-to-edge" diseinua lortzea. Horrek esan nahi du UI-k pantaila osoa erabiltzen duela, aurreko eremuak erreserbatu gabe.

setContent { ... }

- `setContent` barruan idazten duzun guztia aplikazioaren pantailan agertuko da.

HelloWorldTheme { ... }

- Aplikazioaren itxura tematikoa ezartzen du (koloreak, tipografia, etab.).
- Normalean Theme.kt fitxategian definituta dago.

Scaffold

- Jetpack Compose-ko `layout nagusi` bat da.
- `modifier = Modifier.fillMaxSize()` esaten duenez, pantaila osoa hartuko du.
- `{ innerPadding -> ... }` parametroan, Scaffold-ek uzten duen padding-a pasatzen zaio edukiei.

Greeting(...)

- Greeting izeneko `Composable` bat deitzen du.
- `name = "Android"` pasatzen dio, hau da, mezu pertsonalizatu bat erakusteko erabiliko du.
- `modifier = Modifier.padding(innerPadding)` Edukia Scaffoldeko barne-paddinga errespetatuz margotuko da, ez da sistemako barra azpian moztuko
- `@Composable` Composerri esaten dio funtzio hau ez dela arrunta, baizik eta UI marrazteko dela. Barruan beste Composable batzuk erabili ditzakezu (**Text**, **Column**, **Row**, **Button**... adibidez) pantailak eraikitzeko.

```
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}
```

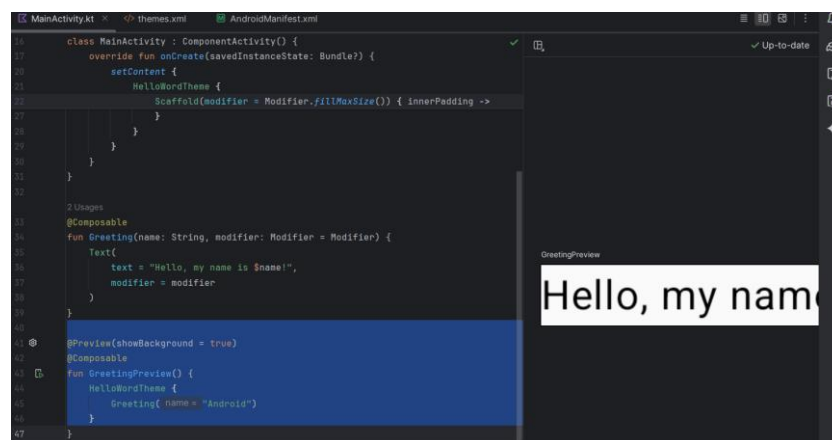
Greeting() funtzioa. **@Composable** oharrarekin dator. Konposagarritasun-funtzio batek sarreraren bat hartzen du eta pantailan erakusten dena sortzen du.

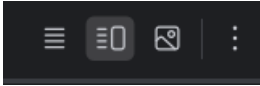
Oraingoz, **Greeting()** funtzioak izen bat hartzen du eta pertsona horri zuzendutako Hello agurra erakusten du. Eguneratu **Greeting()** funtzioa, "Hello" erakutsi beharrean ondorengoa aurkezteko.

Primeran! Testua aldatu da, baina Android gisa aurkeztzen zaituzte, ziurrenik zurea ez den izen bat. Ondoren, aurkezpena zure izenarekin pertsonalizatuko dugu.

DefaultPreview () funtzioa bikaina da, eta aplikazioa erabat konpilatu gabe nola ikusten den ikusteko aukera ematen dizu. Aurretiko bistaren funtzioa izan dadin, **@Preveiw** ohar bat gehitu behar duzu.

Ikus daitekeenez, **@Preveiw** oharrak showBackground izeneko parametro bat hartzen du. ShowBackground egiazkotzat jotzen baduzu, atzealdea erantsiko zaio zure app-aren aurrebistari.

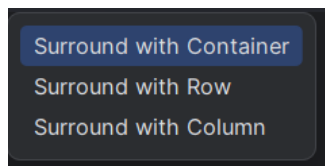
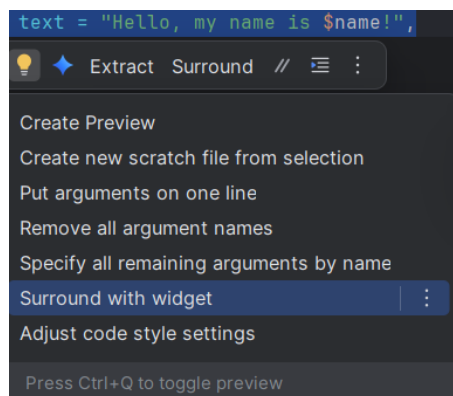




Hiru aukera hauetan codigoa, codigoa+preview eta preview-a ikusteko aukerak ematen ditu.

ARIKETATXOA:

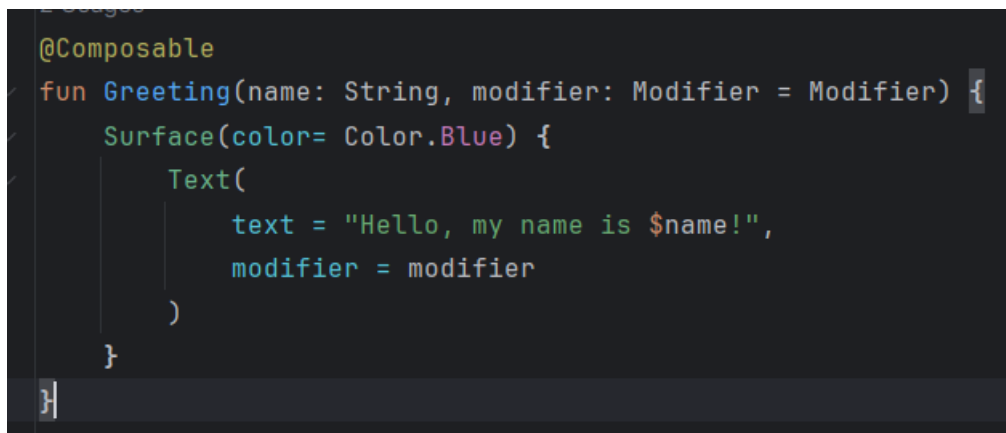
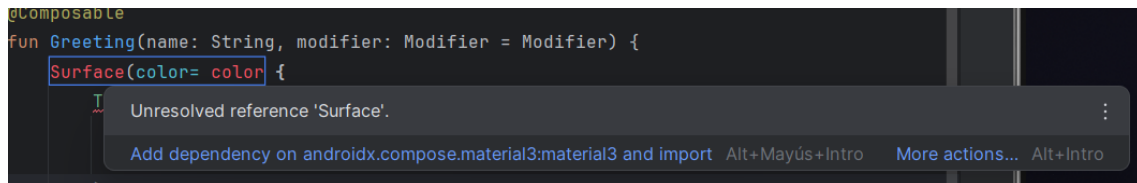
Aurkezpenerako atzealdeko kolore desberdina ezarri nahi izanez gero, testua **surface** batean bildu beharko da. **Surface** bat IUren sekzio bat irudikatzen duen edukiontzi bat da, eta bertan itxura alda dezakezu, hala nola ertza edo hondoko kolorea. Testua Surface batekin inguratzeke:



```
2 Usages
@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Box {
        Text(
            text = "Hello, my name is $name!",
            modifier = modifier
        )
    }
}
```

Ezabatu `Box()` eta idatzi `Surface ()` bere orde. `Surface` edukiontzia koloretako parametro bat du. Konfiguratu kolore gisa. Ondorengo itzura izan behar du:

Bia `Surface` eta bai `color` idazten duzunean, hitza gorri eta azpimarratuta egongo da. Konpontzeko:



Orain zuen testuaren atzealdean urdin kolorea du; jarraian, espazio pixka bat (padding) gehituko dugu testuaren inguruan.

`Modifier` bat erabiltzen da konposizioa onartzen duen elementu bat handitzeko edo apaintzeko. `Padding` aldatzailea erabil daiteke, elementuaren inguruan espazioa gehitzeko. `Modifier.padding ()` funtzioaren bidez lortzen da hori. Beharrezko importazioak gehitu.

